1 Lesson 06-11-26

1.1 MOV instructions in ARM

In assembly ARM we have a number of instructions to load literals into registers, worth seeing.

- The usual `mov` is `mov <rd>, <rn> {, shift}`. `shift` is an optional shift applied to `Rn`:
  - `ASR #n` : arithmetic shift right;
  - `LSL #n` : logical shift left;
  - `LSR #n` : logical shift right;
  - `ROR #n` : rotate right;
  - `RRX` : rotate right 1 bit with extend.
- There is also a way to move a constant, with `mov <Rd>, #constant`. The constant is stored on 8 bits so there are limitations. It can be:
  - A value obtained by shifting left an 8-bit value up to 24 positions;
  - A value of the form `0x00XY00XY`;
  - A value of the form `0xXY00XY00`;
  - A value of the form `0xXYXYXYXY`.
- `mvn` has the same exact syntax as `mov`, but moves the one's complement of the operand into a register;
- `movw` is the wide equivalent of `mov`. It moves a 16-bit value in the low halfword of a register. The assembler can replace `mov` with `mvn` or `MOVW` if needed. This increases the number of possible values supplied as constants to `mov`:
  - A 16-bit value: 0-65535 (`mvw`),
  - A value obtained by shifting left an 8-bit constant up to 24 positions;
  - A value obtained by shifting left an 8-bit constant up to 24 positions, and then applying a bitwise logical NOT operation (`mvn`);
  - A value of the form `0x00XY00XY` or `0xFFXYFFXY` (`mvn`);
  - A value of the form `0xXY00XY00` or `0xXYFFXYFF` (`mvn`);
  - A value of the form `0xXYXYXYXY`.
- `movt` is the upper 16-bit version of `movw`. It moves a 16-bit value in the high halfword of a register. A register can be set to any 32-bit constant by using `movw` and `movt` together, so the pseudo-instruction `mov32` is translated into a combination of those.

1.1.1 LDR for constants

Besides loading values from memory (also note that `mov` can't load value adderesses), `LDR` can be used to load constants into registers: `ldr <Rd>, =<constant>`.

It is a pseudo-instruction:

- If constant is among the valid values of MOV, then the instruction is replaced with: `mov <Rd>, #<constant>`;
- Otherwise, a block of constants, called **literal pool**, is created and the instruction becomes: `ldr <Rd>, [PC, #<offset>]`.